
Multithreading & Synchronization

Concepts and Programming in C, Python, and Java

Intended audience: B.Tech (4th Sem, TU CSE)

Learning outcomes

Understand threads vs processes and thread lifecycle

Explain race conditions, mutexes, semaphores, and thread safety

Apply concepts with examples in C, Python, and Java

Why multithreading?

Modern CPUs provide multiple cores; multithreading helps exploit CPU parallelism and concurrency

Threads can perform tasks like UI responsiveness, background I/O, and parallel computations

Compared to multiple processes, threads are cheaper to create and switch because they share the same address space

Process vs Thread

Process

- Program in execution with its own separate address space.
- Possesses its own independent OS resources (file descriptors, memory).

Thread

- Lightweight execution unit inside a process.
- Shares code, data, and heap with peer threads.
- Maintains its own separate stack and CPU registers.

Implications of sharing

Threads can read/write the same variables directly â†’ fast communication

Danger: unsynchronized access causes races and inconsistent states

Correctness requires synchronization to control critical sections

Basic multithreading model & scheduling

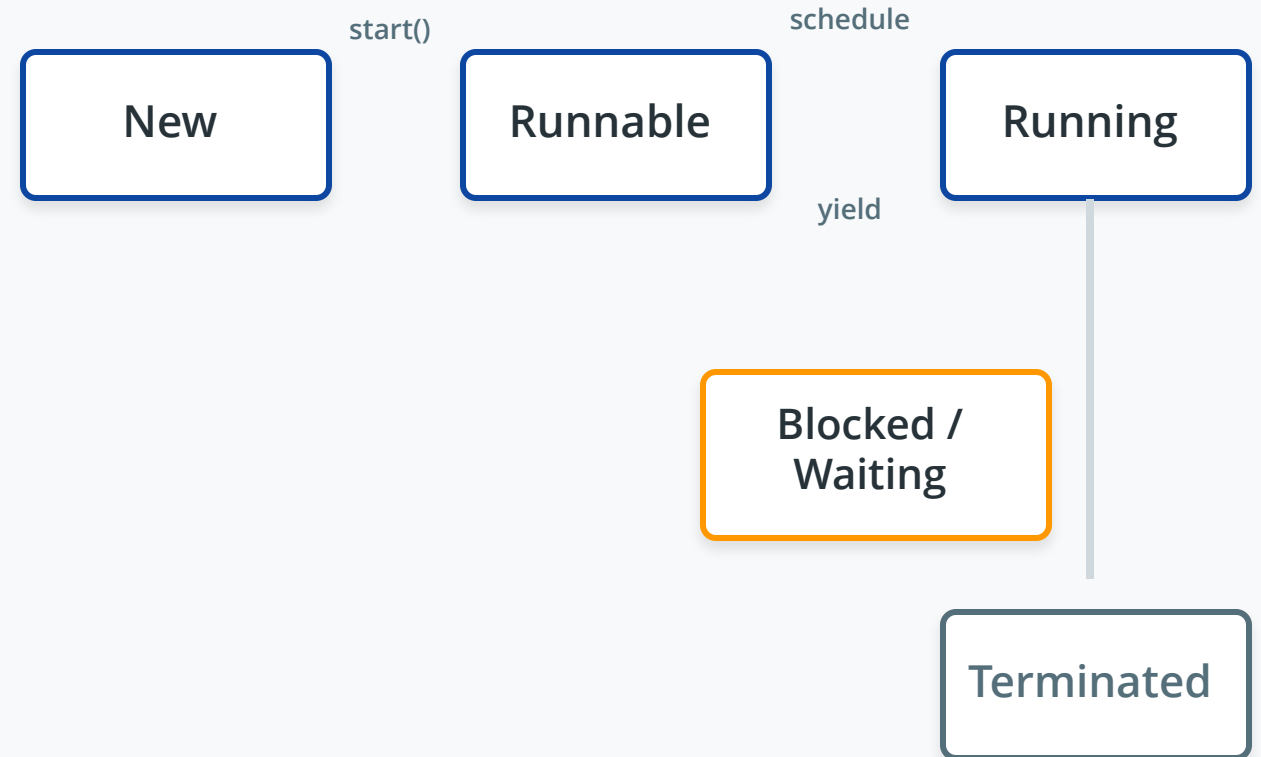
Singleâ€™process, multiâ€™thread model: multiple threads execute in the same virtual address space.

Threads are scheduled by the OS; each thread can be in states like ready, running, blocked, or terminated.

Concurrency: even on a single core, OS switches threads rapidly to give illusion of simultaneous execution.

Thread lifecycle “conceptual”

- **States:** New → Runnable (ready) → Running → Blocked/Waiting → Runnable → Terminated
- **New → Runnable:** thread created, `start()` (or equivalent) called
- **Running → Blocked:** waiting for lock, I/O, sleep; returns to Runnable when condition satisfied



Thread lifecycle by language

We'll map the conceptual lifecycle to Java, POSIX threads (C), and Python threading (with GIL specifics)

Java thread lifecycle

States: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

start() moves NEW → RUNNABLE; scheduler decides when it actually RUNS

run() completes → TERMINATED; cannot restart

Java example “ creating and starting a thread

• • • Demo.java

```
class MyTask extends Thread {
    public void run() {
        System.out.println("Hello from " + Thread.currentThread().getName());
    }
}

public class Demo {
    public static void main(String[] args) {
        Thread t = new MyTask(); // NEW
        t.start();                // RUNNABLE → RUNNING
    }
}
```

C (pthreads) lifecycle & APIs

POSIX threads (`pthread_t`) are native OS threads; their lifecycle parallels processes.

Key APIs: `pthread_create`, `pthread_join`, `pthread_exit`.

Conceptually similar lifecycle: threads can be created, runnable, blocked on synchronization or I/O, or terminated.

C (pthreads) â€™ minimal example

```
demo.c

#include <pthread.h>
#include <stdio.h>

void* worker(void* arg) {
    printf("Hello from thread\n");
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, worker, NULL);
    pthread_join(t, NULL); // wait for termination
    return 0;
}
```

Python threading & the GIL

`threading.Thread` maps to native OS threads in CPython

Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time

I/O-bound threads can run effectively (GIL released during blocking I/O)

Python â€™ minimal example

demo.py

```
import threading

def worker():
    print("Hello from", threading.current_thread().name)

t = threading.Thread(target=worker)
t.start()
t.join()
```

Creating threads “ patterns

Across C, Python, and Java: define work ‘ create thread ‘ start ‘
wait (join)

Creating threads “side-by-side” (C / Python / Java)

Notice the common pattern across languages: define work + start + join (wait)

```
C (pthreads)

// 1. Define target (worker
func)
// 2. Create and start
pthread_t t;
pthread_create(&t, NULL,
worker, NULL);

// 3. Wait for completion
pthread_join(t, NULL);
```

```
Python

# 1. Define target (worker
func)
# 2. Create thread
t =
threading.Thread(target=worker)

# 3. Start execution
t.start()

# 4. Wait for completion
t.join()
```

```
Java

// 1. Define target (Runnable)
// 2. Create thread
Thread t = new Thread(() -> {
    System.out.println("Hello");
});

// 3. Start execution
t.start();

// 4. Wait for completion
t.join();
```

All three implementations logically perform: Define Work → Create → Start → Wait (Join)

Shared data and critical sections

Shared data: variables, objects, files, or data structures accessed by multiple threads.

Critical section: part of code where such shared variables are read or modified; final result depends on execution order of threads.

If multiple threads enter a critical section simultaneously without control, we may get inconsistent results.

Race conditions “definition”

Bug where outcome depends on timing/interleaving of threads in a critical section

Common patterns: **read** “**modify**” **write** (++, --);
check “**then**” **act** (if not null then use)

Synchronization aims to eliminate data races and ensure determinism under scheduling variability

Race condition “simple scenario”

Shared counter = 0; two threads increment 10000 times each

Expected: 20000; Actual: often less (increments are not atomic)

Interleaving example: both read old value, both compute new, last write wins

Use a timeline diagram on delivery to illustrate interleavings

Race condition code “ C example

Ask students: run multiple times and observe non-deterministic output.

```
race_demo.c

#include <pthread.h>
#include <stdio.h>
int counter = 0; // shared global
void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; // NOT ATOMIC → race condition
    }
    return NULL;
}
int main() {    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Counter = %d\n", counter); // often < 200000
    return 0;
}
```

Race condition code “ Python example

● ● ● race_condition.py

```
import threading
counter = 0
def increment():
    global counter
    for _ in range(100000):
        counter += 1 # read-modify-write, not atomic
threads = [threading.Thread(target=increment) for _ in range(2)]
for t in threads: t.start()
for t in threads: t.join()
print("Counter =", counter)
```

Note: Even with the GIL, Python’s counter += 1 is not guaranteed to be atomic across threads.

Race condition code “ Java example

```
RaceDemo.java

class Counter {
    public int value = 0;
}

public class RaceDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Runnable task = () -> {
            for (int i = 0; i < 100000; i++) c.value++; // not atomic
        };
        Thread t1 = new Thread(task), t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("Counter = " + c.value);
    }
}
```

Note: Java™s `c.value++` is a non-atomic read-modify-write.

Synchronization “goals & techniques”

Goals: mutual exclusion, ordering constraints, and visibility of updates across threads

Techniques:

- Locks/mutexes (mutual exclusion objects)

- Semaphores (counting and binary)

- Condition variables / wait‘notify, barriers

- Atomic variables and lock‘free algorithms

Mutex “concept”

Mutual exclusion object to protect a critical section

Acquire (lock) before entering; release (unlock) on exit

Typically, the thread that locks must unlock; others block until available

Semaphore “concept”

Integer used for signaling and resource pool control

wait/P/down: decrement; **signal/V/up:** increment

Types: counting (N resources), binary (0/1; similar to mutex semantics)

Mutex vs Semaphore “comparison

Aspect	Mutex	Semaphore
Mechanism	Locking	Signaling
Type	Object	Integer counter variable
Ownership	Typically only owner thread unlocks	Any thread may signal
Count	Only one owner at a time	Counting or binary; can allow multiple simultaneous holders
Use Case	Exclusive access to a single shared resource	Limit concurrent access to N instances (e.g., N connections)

Mutex example “ C (pthreads)

```
increment.c

#include <pthread.h>
#include <stdio.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Mutex example “ Java (synchronized)

SafeCounter.java

```
class SafeCounter {  
    private int value = 0;  
    public synchronized void inc() { value++; } // protected critical  
section  
    public int get() { return value; }  
}
```

Note: synchronized methods/blocks provide mutual exclusion and visibility guarantees.

Semaphore example in C (POSIX)

Pattern: limit concurrency to available slots.

```
worker.c

#include <semaphore.h>

sem_t slots;

void* worker(void* arg) {
    sem_wait(&slots);    // acquire a slot
    // do work
    sem_post(&slots);    // release slot
    return NULL;
}
```

Semaphore example “ Java

SemaphoreDemo.java

```
import java.util.concurrent.Semaphore;

Semaphore slots = new Semaphore(3); // at most 3 concurrent workers

Runnable task = () -> {
    try {
        slots.acquire();
        // limited resource work
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        slots.release();
    }
};
```

Python synchronization â€” Lock and Semaphore

sync_demo.py

```
import threading
counter = 0
lock = threading.Lock()
def increment():
    global counter
    for _ in range(100000):
        with lock:          # acquire, then release
            counter += 1

sem = threading.Semaphore(3)

def limited_task():
    with sem:                # at most 3 such functions running simultaneously
        # do limited work
    pass
```

Deadlocks, livelocks, starvation

Deadlock: a set of threads permanently blocked, each waiting for a resource held by another in the set (circular wait).

Livelock: threads keep changing state in response to others but never make progress (e.g., repeatedly yielding to each other).

Starvation: a thread never gets CPU or lock access because others keep beating it to the resource (e.g., unfair scheduling).

Prevention tip: acquire multiple locks in a consistent global order to avoid circular wait and potential deadlocks.

Thread-safe programming & strategies

Thread-safe:

correct behavior under concurrent execution; no data races; visibility ensured.

Avoid sharing:

use thread-local variables or pass copies instead of sharing objects.

Immutability:

immutable objects can be freely shared across threads without locks.

Confinement:

restrict object access to a single thread (e.g., UI thread confinement).

When sharing mutable state:

use locks, atomics, or specialized thread-safe collections/abstractions.

Higher-level concurrency utilities

Java:

java.util.concurrent provides thread pools (ExecutorService), concurrent collections, locks (ReentrantLock), and atomics (AtomicInteger).

Python:

concurrent.futures.ThreadPoolExecutor encapsulates worker threads and task submission.

C (Libraries):

Abstraction is often library-specific (e.g., OpenMP, TBB) built on top of pthreads.

```
ExecutorService pool = Executors.newFixedThreadPool(4);  
pool.submit(() -> doWork());  
pool.shutdown();
```

Memory visibility: GIL vs Java Memory Model

CPython GIL: protects interpreter state; does not make compound operations on objects logically atomic

Java Memory Model: defines visibility/order; synchronized, volatile, and final provide happens-before guarantees

Without visibility guarantees, threads may read stale data even if no race seems apparent

Debugging concurrency + best practices

Symptoms

Intermittent failures, nonâ€™reproducible outputs, hangs, and data corruption

Tools & Techniques

Logging with thread IDs, IDE debuggers/profilers, static analyzers, small test loads to reproduce races

Best Practices

- â€™ Minimize shared state; use thread pools instead of raw threads
- â€™ Keep critical sections small and clearly define resource ownership
- â€™ Document invariants; avoid nested locks or acquire in fixed order to prevent deadlocks

Producerâ€™Consumer (Bounded Buffer)

We will implement producerâ€™consumer using:

â€¢ **C** (pthread + mutex + condition variables)

â€¢ **Java** (synchronized + wait/notify)

â€¢ **Python** (queue.Queue and Condition variant)

Producerâ€™Consumer in C (pthread) â€™ part 1

```
producer_consumer_pthreads.c

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0, count = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // produce an item
        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&not_full, &mutex);
        }
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        pthread_cond_signal(&not_empty);
        sleep(1); // simulate production time
        printf("Produced: %d at %d\n", item, in);
        count++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Producerâ€™Consumer in C (pthread) â€™ part 2

```
producer_consumer.c

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mutex);
        }
        int item = buffer[out];
        printf("Consumed: %d at %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        count--;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

int main(void) {
    pthread_t prod, cons;
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);
    return 0;
}
```

Key points: Buffer indices/count are protected by a mutex. `not_full` and `not_empty` condition variables implement blocking behavior instead of busyâ€™waiting.

Producer-Consumer in Java (wait/notify) part 1

ProducerConsumer.java

```
// ProducerConsumer.java (excerpt)
import java.util.LinkedList;
import java.util.Queue;
public class ProducerConsumer {
    private static final int CAPACITY = 5;
    private final Queue<Integer> buffer = new LinkedList<>();
    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();
        Thread producer = new Thread(pc.new Producer(), "Producer");
        Thread consumer = new Thread(pc.new Consumer(), "Consumer");
        producer.start(); consumer.start();
    }
    class Producer implements Runnable {
        private int value = 0;
        public void run() {
            try { while (true) { produce(value++); Thread.sleep(500); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
        private void produce(int item) throws InterruptedException {
            synchronized (buffer) {
                while (buffer.size() == CAPACITY) { buffer.wait(); }
                buffer.add(item);
                System.out.println("Produced: " + item);
                buffer.notifyAll();
            }
        }
    }
}
```


Producerâ€™Consumer in Java (wait/notify) â€™ part 2

```
Consumer.java

class Consumer implements Runnable {
    public void run() {
        try {
            while (true) {
                int item = consume();
                System.out.println("Consumed: " + item);        Thread.sleep(800);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    private int consume() throws InterruptedException {
        synchronized (buffer) {
            while (buffer.isEmpty()) { buffer.wait(); }
            int item = buffer.remove();        buffer.notifyAll();
            return item;
        }
    }
}
```

Key points:wait() / notifyAll() must be called inside synchronized on the same monitor; whileâ€™loops handle spurious wakeups.

Producerâ€™Consumer in Python â€™ Queue variant (part 1)

```
producer_consumer_queue.py

# producer_consumer_queue.py (excerpt)
import threading, time, random
from queue import Queue
BUFFER_SIZE = 5
queue = Queue(maxsize=BUFFER_SIZE)
def producer():
    item = 0
    while True:
        time.sleep(random.uniform(0.2, 1.0))
        queue.put(item) # blocks if queue is full
        print(f"Produced: {item} (size={queue.qsize()})")
        item += 1
```

Producerâ€™Consumer in Python â€™ Queue variant (part 2)

```
producer_consumer_queue.py

def consumer():
    while True:
        item = queue.get() # blocks if queue is empty
        print(f"Consumed: {item} (size={queue.qsize()})")
        time.sleep(random.uniform(0.5, 1.2))
        queue.task_done()

if __name__ == "__main__":
    t_prod = threading.Thread(target=producer, daemon=True)
    t_cons = threading.Thread(target=consumer, daemon=True)
    t_prod.start(); t_cons.start()
    time.sleep(10)
    print("Main thread exiting...")
```

Note: Queue is threadâ€™safe; handles locking and signaling internally. See also Conditionâ€™based variant for OSâ€™theory alignment.

Producerâ€™Consumer in Python â€™ Condition variant

Shared Setup

Shared buffer of size N; `in_idx` / `out_idx` / `count`

```
mutex = Lock(); not_full = Condition(mutex); not_empty = Condition(mutex)
```

Producer flow (with `not_full`)

```
with not_full:
    while count == N: not_full.wait()
    insert item; count += 1; not_empty.notify()
```

Consumer flow (with `not_empty`)

```
with not_empty:
    while count == 0: not_empty.wait()
    remove item; count -= 1; not_full.notify()
```

Note:

Mirrors C pthreads logic with conditions; ideal for bounded buffer semantics.

Wrap-up & exam-style questions

Recap

- **Threads vs processes** and thread lifecycle across Java, C (pthreads), and Python.
- **Race conditions**, critical sections, and the need for controlled access to shared state.
- Synchronization mechanisms using **mutexes** and **semaphores**.
- Basics of **thread-safe design** and avoiding deadlocks.
- Practical **producer-consumer** implementations.

Sample Questions

1. Explain, with a diagram, the Java thread lifecycle, including the states and methods that trigger transitions.
2. Give a code example of a race condition and demonstrate how to fix it using a mutex in a language of your choice.
3. Compare and contrast a mutex and a semaphore. Provide an example scenario where a counting semaphore is preferred over a mutex.